Dr. D. Y. Patil Educational Federation's

Dr. D. Y. Patil College of Engineering & Innovation

Approved by AICTE, Recognized by Government of Maharashtra

An Autonomous Institute Affiliated to Savitribai Phule Pune University

Accredited by NAAC with "A" Grade

DEPARTMENT OF COMPUTER ENGINEERING

NASSCOM PROJECT REPORT

PERSONAL FINANCE ADVISOR BOT

AI-Powered Personal Budgeting and Savings Assistant

Program / Class	S.Y. B.Tech
Division / Batch	A
Name of Student	Harsh vilas mastud
Roll Number	12152
Project Category	Finance – Individual Project
Project Progress	100% (as provided in project overview)
Academic Year	2026–2027

Submitted as a project report for NASSCOM project documentation.

1. Project Description

1.1 Overview

The Personal Finance Advisor Bot is an AI-powered financial planning assistant designed to help individuals manage their personal finances with greater clarity and confidence. It provides a centralized environment in which users can record monthly income, log daily and weekly expenses by category, receive personalized budget plans, and track their savings progress.

The platform combines backend technologies with an AI-driven financial analysis engine to support budget generation, spending insights, and structured monthly financial reporting. By reducing the need for manual calculations and scattered records, the system aims to make personal finance tracking more organized and easier to maintain.

The described backend architecture uses Flask and SQLAlchemy. The system is intended to support future enhancements such as predictive spending analytics, investment-related suggestions, goal-based savings tracking, and advanced financial-health monitoring. The exact AI model, frontend technology, database engine, and deployment configuration were not specified in the supplied project description.

1.2 Problem Statement

Individuals often find it difficult to maintain a consistent record of income and expenses, understand where their money is going, and set realistic budgets. Manual tracking can become time-consuming, and users may not notice overspending until the end of the month. A centralized assistant that organizes financial entries and provides understandable budget and saving insights can help users develop better financial habits.

1.3 Aim

To develop an AI-powered personal finance assistant that helps individuals record income and expenses, plan budgets, monitor savings, and understand spending patterns through structured financial insights.

1.4 Objectives

Provide a centralized way to record monthly income and daily or weekly expenses.
Organize expenses into categories for easier review and analysis.
Generate personalized budget plans based on the financial information entered by the user.
Identify spending categories where expenses may be higher than planned.
Track saving progress and present monthly summaries of income, expenses, and savings.
Offer practical saving recommendations to support informed personal budgeting.
Provide a foundation for future features such as predictive analytics and goal-based savings.

2. Technology and AI

The following technologies are stated in the supplied project description. Details not provided are identified as unspecified.

Component	Technology / Description
Backend / Web Framework	Python with Flask
Database Interaction / ORM	SQLAlchemy
AI / Analysis	AI engine for financial analysis; exact model or API not specified
Database Engine	Not specified in the supplied description
Frontend	Not specified in the supplied description
Version Control / Repository	Repository link not included in the supplied description

3. Key Features

Income recording: allows users to enter their monthly income.
Expense tracking: supports recording daily and weekly expenses.
Expense categorization: groups expenses into categories to make spending patterns easier to understand.
Personalized budget planning: uses the financial information entered by the user to prepare a budget plan.
Spending insights: highlights categories that may require attention or where spending may be excessive.
Savings progress tracking: helps users review how much they are saving.
Monthly financial reporting: presents a structured summary of income, expenses, and savings.
Saving recommendations: provides practical suggestions intended to help users reduce unnecessary expenditure.
Extensible architecture: leaves room for future improvements such as predictive spending analytics and goal-based savings.

4. Scenarios

Scenario 1: Budget Planning for a Salaried Professional

A salaried professional records monthly income and daily expenses for rent, food, transport, entertainment, and other categories. The system reviews the entered spending data, highlights categories with possible overspending, and generates a personalized budget plan with saving suggestions. At the end of the month, the user can review a summary of income, expenses, savings achieved, and financial goals for the next month.

Scenario 2: Allowance Management for a College Student

A college student uses the bot to manage a limited monthly allowance. The student sets budget limits for essential and discretionary categories and records expenses as they occur. The platform helps the student review spending habits and receive AI-powered suggestions for reducing unnecessary expenditure, supporting better financial discipline and savings management.

Scenario 3: Variable Income Management for a Freelancer

A freelancer records earnings from multiple clients and tracks project-related expenses. Because income may vary from month to month, the system can help the user review spending against the available income, identify periods with lower saving capacity, and consider strategies for building an emergency fund. Structured tracking supports more informed long-term money management.

Scenario 4: Household Budget Management

A household manager tracks shared family income and expenses such as groceries, utilities, education, and healthcare. The bot provides a consolidated monthly summary, highlights categories where spending may exceed the planned budget, and suggests saving strategies aligned with family goals. This can support household planning and resource allocation.

5. Technical Architecture

Based on the supplied description, the application uses a Flask-based backend, SQLAlchemy for database interaction, and an AI engine for financial analysis. The user-facing interface and specific database engine were not identified in the provided information, so they are not named in the architecture below.

User Input	Flask Backend	SQLAlchemy / Data Layer	AI Analysis Engine
Income, expenses, categories, budget preferences	Receives requests and coordinates application logic	Supports storage and retrieval of financial records	Analyzes financial information and supports insights

General data flow: the user enters financial information → the Flask application processes the request → SQLAlchemy supports access to stored records → the AI analysis component supports budget insights and saving recommendations → the system presents results and summaries to the user. This is a conceptual flow based only on the supplied overview; implementation-specific details should be checked against the project repository.

6. System Requirements

6.1 Hardware Requirements

Computer or laptop suitable for Python development.
Adequate memory and storage to run the application and its development tools.
Internet connection if the AI engine or any required service is accessed online.

6.2 Software Requirements

Operating system compatible with the project's Python environment.
Python installed in a supported version for the project.

Flask and the project's required Python packages.
SQLAlchemy and the database driver appropriate to the actual database configuration.
A code editor or IDE, such as Visual Studio Code or an equivalent tool.
AI service credentials or local AI dependencies, if required by the actual implementation.
A web browser if the project provides a browser-based interface.
Note: Exact versions, database engine, browser requirements, and installation commands should be taken from the project's README or dependency files. They were not supplied in the project description.

7. General Usage Instructions

Obtain the project source code from the project repository.
Review the README and dependency files to identify the supported Python version and setup procedure.
Create and activate a Python virtual environment if recommended by the project instructions.
Install the project dependencies using the commands specified in the repository.
Configure the database connection and any required AI settings according to the actual project configuration.
Start the Flask application using the run instructions provided by the project.
Open the application's interface and enter monthly income and expense records.
Review category-wise spending, budget guidance, savings progress, and monthly summaries where these features are available.
Test the application with valid and invalid entries and confirm that financial records and summaries behave as expected.

8. Milestones

Milestone 1 – Requirement Analysis: Identify target users, financial information to be recorded, and the key budgeting and savings workflows.
Milestone 2 – Data and Category Design: Plan how income, expense entries, categories, budget limits, and monthly summaries are represented.
Milestone 3 – Backend Development: Use Flask to implement application routes and business logic according to the project design.
Milestone 4 – Data Persistence: Use SQLAlchemy to support database operations for relevant financial records.
Milestone 5 – AI-Based Financial Analysis: Connect the AI analysis component to the financial information needed for budget insights and saving recommendations.
Milestone 6 – Testing and Validation: Check expense entry, category handling, budget generation, savings calculations, and monthly reporting.
Milestone 7 – Final Review: Review the user experience, verify the documented setup instructions, and prepare the project for demonstration.

9. Expected Outcome

The expected outcome is a centralized personal finance assistant that helps users record income and expenses, organize spending by category, review budget guidance, and monitor savings progress. Monthly summaries are intended to make financial activity easier to understand, while AI-supported insights can help users identify areas where spending may need attention. The quality of recommendations depends on the accuracy and completeness of the data entered and on the capabilities of the configured AI engine.

10. Conclusion

The Personal Finance Advisor Bot is designed to simplify personal budgeting by combining income and expense tracking, category-based spending review, budget generation, savings monitoring, and AI-supported recommendations in one digital environment. Its Flask and SQLAlchemy-based backend provides the stated technical foundation for managing application logic and financial records. The four use scenarios demonstrate how the system can support salaried professionals, students, freelancers, and household managers. Future improvements may include predictive spending analytics, goal-based savings tracking, and additional financial-health features.

11. Scope and Responsible Use

The bot is intended to support personal budgeting and financial awareness. Its suggestions should be treated as informational assistance rather than a guarantee of financial outcomes. Users should protect their financial data, avoid entering unnecessary sensitive information, and review recommendations before acting on them. Investment-related suggestions are described as a possible future enhancement and should not be represented as an implemented feature unless confirmed by the project source code.

Project Information Checklist

The following details were not included in the supplied description and should be confirmed before final submission:

Mentor name and email address.
Project repository and demo URLs, if they should appear in the report.
Exact AI model or API and the frontend technology, if applicable.
Database engine and confirmed implementation details.
Actual setup commands and supported software versions.